

§ SOMMAIRE

Table des matières

01	Introduction à la blockchain	Définitions, propriétés fondamentales	p.02
02	Fonctionnement technique	Blocs, hachage, consensus, types de réseaux	p.03
03	Smart contracts — fondamentaux	EVM, Solidity, cycle de vie d'un contrat	p.04
04	Développer un smart contract	Structure Solidity, standards ERC-20/721	p.05
05	Sécurité des smart contracts	Vulnérabilités courantes, audits	p.06
06	Écosystème & cas d'usage	DeFi, NFT, DAO, Layer 2	p.07
07	Outils & ressources	Environnements de développement, pour aller plus loin	p.08

Qu'est-ce que la blockchain ?

Une **blockchain** est un registre distribué qui enregistre des transactions de façon **chronologique, vérifiable et quasiment impossible à falsifier**, sans autorité centrale. Chaque bloc contient un ensemble de transactions et une empreinte cryptographique (hash) du bloc précédent, formant une chaîne où toute modification rétroactive briserait l'ensemble de la structure.

Propriétés fondamentales

- **Décentralisation** — aucun point de contrôle unique
- **Immutabilité** — historique quasi impossible à réécrire
- **Transparence** — transactions vérifiables publiquement
- **Consensus** — accord collectif sur l'état du registre

Types de blockchains

Type	Accès
Publique	Ouverte à tous (Bitcoin, Ethereum)
Privée	Contrôlée par une seule organisation
Consortium	Gérée par un groupe d'organisations

À retenir — La blockchain n'est pas une technologie unique mais une combinaison de concepts existants — cryptographie à clé publique, arbres de Merkle, systèmes distribués — assemblés en 2008 par Satoshi Nakamoto dans le livre blanc de Bitcoin.

§ MODULE 01

Comment fonctionne une blockchain ?

Chaque transaction diffusée sur le réseau est regroupée avec d'autres dans un **bloc**, validée par les participants selon un mécanisme de consensus, puis ajoutée de façon définitive à la chaîne.

01

Transaction

Un utilisateur signe une transaction avec sa clé privée.

02

Diffusion

La transaction est propagée à l'ensemble des nœuds du réseau.

03

Validation

Les nœuds vérifient signature, solde et règles de consensus.

04

Bloc

Les transactions validées sont regroupées dans un nouveau bloc.

05

Hachage

Le bloc est scellé par un hash incluant celui du bloc précédent.

∞

Chaîne

Le bloc rejoint définitivement l'historique partagé.

Mécanismes de consensus

Mécanisme	Principe	Exemple
Proof of Work (PoW)	Résolution d'un puzzle cryptographique coûteux en calcul (minage)	Bitcoin
Proof of Stake (PoS)	Validation proportionnelle aux jetons mis en jeu (staking)	Ethereum (depuis 2022)
Delegated PoS	Élection d'un nombre restreint de validateurs délégués	Certaines sidechains

Structure d'un bloc

```
● ● ● block.json

{
  "index": 4821,
  "timestamp": 1732012345,
  "prevHash": "0x8b1d...",
  "transactions": [ ... ],
  "nonce": 918273,
  "hash": "0x4f3a..."
}
```

Fonction de hachage

Une fonction comme **SHA-256** ou **Keccak-256** transforme n'importe quelle donnée en une empreinte de taille fixe. La moindre modification du bloc change entièrement son hash, ce qui rend toute falsification immédiatement détectable par le réseau.

« Modifier une transaction ancienne obligerait à recalculer le hash de tous les blocs suivants, sur la majorité des nœuds — un coût pratiquement prohibitif. »

§ MODULE 02

Smart contracts — les fondamentaux

Un **smart contract** (contrat intelligent) est un programme stocké sur la blockchain qui s'exécute automatiquement lorsque ses conditions sont remplies, sans intermédiaire. Le concept a été théorisé par Nick Szabo dans les années 1990, mais popularisé par **Ethereum** à partir de 2015.

L'Ethereum Virtual Machine (EVM)

L'**EVM** est l'environnement d'exécution standardisé qui fait tourner le bytecode des smart contracts de façon identique sur tous les nœuds du réseau. Chaque opération y consomme du **gas**, une unité qui mesure le coût de calcul et évite les boucles infinies.

Cycle de vie d'un contrat

- Écriture du code (Solidity, Vyper...)
- Compilation en bytecode EVM
- Déploiement via une transaction (adresse générée)
- Appels de fonctions par d'autres transactions
- Exécution déterministe et vérifiable par tous

Le gas, en bref

Concept	Rôle
Gas limit	Quantité maximale de calcul autorisée
Gas price	Prix payé par unité de gas (gwei)
Gas used	Calcul réellement consommé

Premier contrat en Solidity

```
● ● ● Storage.sol

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract Storage {
    uint256 private value;

    function set(uint256 _value) public {
        value = _value;
    }

    function get() public view returns (uint256) {
        return value;
    }
}
```

À retenir — Une fois déployé, le code d'un smart contract est **public et immuable** par défaut : toute erreur logique reste exploitable tant qu'un mécanisme de mise à jour (proxy, gouvernance) n'a pas été prévu dès la conception.

§ MODULE 03

Développer un smart contract

Éléments clés du langage Solidity

Élément	Rôle
state variables	Données stockées de façon permanente sur la blockchain
function modifiers	Conditions réutilisables appliquées avant l'exécution d'une fonction
events	Journalisation consultable par les applications externes
mapping	Structure clé-valeur, souvent utilisée pour les soldes
modifiers visibility	public, private, internal, external

Modifier & événement

```
modifier onlyOwner() {
    require(msg.sender == owner,
        "Not authorized");
    _;
}

event Transfer(
    address indexed from,
    address indexed to,
    uint256 amount
);
```

Standards de tokens (ERC)

Standard	Usage
ERC-20	Jetons fongibles (monnaies, points, gouvernance)
ERC-721	Jetons non-fongibles (NFT), chacun unique
ERC-1155	Standard hybride multi-jetons (fongibles + NFT)
ERC-4626	Coffres-forts (vaults) de rendement standardisés

Exemple : extrait d'un token ERC-20

```
● ● ● Token.sol

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract MyToken is ERC20 {
    constructor() ERC20("MyToken", "MTK") {
        _mint(msg.sender, 1_000_000 * 10 ** decimals());
    }
}
```

Bonne pratique — S'appuyer sur des bibliothèques auditées comme **OpenZeppelin** plutôt que de réimplémenter des standards critiques (tokens, contrôle d'accès, protection contre la réentrance).

§ MODULE 04

Sécurité des smart contracts

L'immutabilité des contrats déployés rend chaque faille potentiellement **irréversible et coûteuse** — plusieurs centaines de millions de dollars ont été perdus historiquement à cause de vulnérabilités connues et documentées.

Vulnérabilité	Description	Risque
Reentrancy	Un contrat externe rappelle la fonction avant la mise à jour de l'état interne	CRITIQUE
Integer overflow/underflow	Dépassement des bornes numériques (largement mitigé depuis Solidity 0.8+)	ÉLEVÉ
Access control manquant	Fonctions sensibles appelables par n'importe quelle adresse	CRITIQUE
Front-running	Un tiers observe une transaction en attente et la devance	MOYEN
Oracle manipulation	Falsification des données de prix externes utilisées par le contrat	ÉLEVÉ

Exemple : protection contre la réentrance

✗ vulnérable

```
function withdraw() public {
  (bool ok,) = msg.sender.call{
    value: balance[msg.sender]
  }("");
  balance[msg.sender] = 0; // trop tard
}
```

✓ corrigé (checks-effects-interactions)

```
function withdraw() public {
  uint amt = balance[msg.sender];
  balance[msg.sender] = 0; // avant
  (bool ok,) = msg.sender.call{value: amt}("");
  require(ok);
}
```

Réflexe indispensable — Tout contrat destiné à gérer des fonds réels doit passer par un **audit de sécurité indépendant**, des tests unitaires exhaustifs et idéalement un déploiement progressif (bug bounty, montants plafonnés) avant une mise en production complète.

§ MODULE 05

Écosystème & cas d'usage

◆ DeFi – finance décentralisée

Protocoles permettant prêt, emprunt, échange (DEX) ou épargne **sans intermédiaire bancaire**, régis entièrement par des smart contracts (ex. Uniswap, Aave).

◆ DAO – organisation autonome décentralisée

Structure de gouvernance où les décisions sont proposées et votées **on-chain**, souvent pondérées par des jetons de gouvernance.

◆ NFT – jetons non-fongibles

Représentation unique et vérifiable d'un actif numérique ou physique : art, objets de jeu, billets, titres de propriété.

◆ Traçabilité & supply chain

Suivi immuable de l'origine et du parcours d'un produit, utile pour la certification et la lutte contre la contrefaçon.

Le problème du scaling — solutions Layer 2

Les blockchains publiques comme Ethereum privilégient sécurité et décentralisation au détriment du débit (throughput). Les solutions de **Layer 2** traitent les transactions hors de la chaîne principale avant d'y ancrer un résumé sécurisé.

Solution	Principe
Rollups optimistes	Transactions supposées valides, contestables via une période de challenge
ZK-Rollups	Preuves cryptographiques (zero-knowledge) validant les lots de transactions
Sidechains	Chaînes parallèles avec leur propre consensus, reliées par un pont (bridge)

« Le trilemme de la blockchain (Vitalik Buterin) résume la difficulté à optimiser simultanément décentralisation, sécurité et scalabilité — chaque architecture fait des compromis. »

§ MODULE 06

Outils & ressources

Environnements de développement

Outil	Usage
Remix IDE	IDE en ligne pour écrire, compiler et déployer rapidement des contrats
Hardhat	Framework de développement, tests et déploiement (Node.js)
Foundry	Toolkit rapide en Rust, tests écrits directement en Solidity
OpenZeppelin Contracts	Librairies de contrats standards et sécurisés, largement audités
MetaMask	Portefeuille navigateur pour interagir avec les dApps

Réseaux de test

- Sepolia (testnet Ethereum)
- Réseaux locaux (Hardhat Network, Anvil)
- Faucets pour obtenir des ETH de test

Audit & analyse

- Slither — analyse statique de code
- MythX / Mythril — détection de vulnérabilités
- Programmes de bug bounty (Immunefi)

Pour approfondir

- Documentation officielle Solidity
- Ethereum.org — Developer Docs
- CryptoZombies — apprentissage ludique
- Livre blanc Bitcoin & Ethereum Yellow Paper

En résumé

01 Registre distribué, immuable, sans autorité centrale

02 Consensus PoW / PoS pour valider les blocs

03 Smart contracts = code exécuté par l'EVM

04 Standards ERC-20/721 pour les tokens

05 Sécurité = audits + tests + bonnes pratiques

06 Écosystème riche : DeFi, NFT, DAO, Layer 2

◆ conclusion

La blockchain et les smart contracts ouvrent des possibilités inédites d'automatisation et de désintermédiation, mais exigent une rigueur d'ingénierie supérieure à celle du développement logiciel classique : le code déployé est public, souvent immuable, et manipule fréquemment de la valeur réelle. La pratique sur testnet et l'apprentissage continu des vulnérabilités connues restent essentiels avant tout déploiement en production.

--- block finalized • end_of_course ---

[BUILD SECURE, BUILD TRUSTLESS]